



St. MARTIN'S ENGINEERING COLLEGE

(Autonomous Institution - UGC, Govt. of India)

| ESTD : 2002 | NAAC (A+) & NBA Accredited | Affiliated to JNTUH

| Approved by AICTE | NIRF & ARIIA Ranked | A Non Minority Institution

Dhulapally, Near Kompally, Secunderabad - 500 100. Telangana.



ASSIGNMENT ANSWER BOOKLET

Hall Ticket No. : 24K81A05D3

Name of the Student: A. Deekshitha

Course : B-Tech

Branch : CSE

Year : I/II/III/IV

Sem : I / II

Assignment : I / II

Subject Name : Principles of Programming language.

Last Date of Submission: 25/8/26

Actual Date of Submission : 25/8/26

MARKS AWARDED

Q.No.	a	b	Total
1			
2			
3			
4			
5			
Total			

A. Deekshitha
Signature of the Student

Signature of the Evaluator

Start Writing from Here

1. What is the need for studying concepts of programming language?
A programming language is a formal language used to write instruction and algorithms for a computer. Studying programming language concepts means understanding the basic principles common to different programming languages rather than testing learning only the syntax of one language.
Reasons for studying programming language concepts.
1. Increased capacity to Express Ideas.
The expressive power of a language affects the kinds of ideas and solutions that programmers can express. A programming language places limit on control structures and abstraction that programmers can use. Therefore, the forms of algorithms that can be constructed are also influenced by the language.
2. Improved Background for choosing Appropriate Language. ①



Programmers are often given a choice of language for a new project. They may select the language they are most familiar with, even when another language is better suited to the problem. Knowledge of a wider range of language and language constructs helps programmers select a language with features that best suit the particular problem.

3. Increased ability to Learn new Languages.

Once the fundamental concepts of programming languages are understood, it becomes easy to learn new language. The programmer can relate the concepts of new language to the concepts already known. For Example: A programmer who knows the basic concepts of C can learn Java more easily because both have concepts such as variables, data types, loops & functions.

4. Better understanding of Implementation.

Understanding implementation issues helps programmers know why programming languages are designed in particular ways and use their features more effectively, helps a programmer understand compilation and execution.

5. Better use of Languages Already known.

Programming languages are large and complex, so programmers may not know all their features. Studying programming languages concepts helps them discover and use previously unknown features of languages they already know.

6. Overall advancement of computing.

Studying programming languages concepts helps people make better decisions when choosing and developing programming languages. Popular languages are not always necessarily the best available languages.

b) Compare different programming domains and state their applications.

A programming domain is a particular area in which.



computers are used to solve specific types of problems. Different domains have different requirements, so programming languages have been developed with features suitable for those requirements.

1. Scientific Applications.

Scientific applications were among the earliest uses of digital computers. They mainly require a large number of floating point arithmetic computations. Array and matrices are commonly used as data structures, while counting loops and selections are common control structures.

Example: FORTRAN

Applications: Scientific calculations, Mathematical computations, engineering calculations and scientific research.

2. Business Application.

Business application mainly deals with processing business information. They require facilities for producing elaborate reports, storing decimal numbers and character data and performing decimal arithmetic operations.

Example: COBOL.

3. Artificial Intelligence

Artificial Intelligence is characterized by symbolic rather than numeric computations. Symbolic computations involves manipulating symbols or names rather than mainly performing numerical calculations.

Example: Lisp and Prolog.

4. Web software

Web software is supported by a collection of languages, ranging from markup languages such as HTML to general purpose languages such as Java. Languages such as Javascript and PHP can be used to provide dynamic web content. Applications: Websites, dynamic webpages and web applications.

2. Describe context-free grammar (CFG) and BNF (Backus-Naur Form) with example.

Context-free grammar: A context free grammar is a formal grammar used to describe the syntax of programming ³



languages. It provides a systematic way of specifying how valid sentences and syntactic structures of a language can be formed.

→ Noam Chomsky developed context-free grammar in mid-1950s.

Among the different classes of grammars context-free & regular grammars are useful for describing programming languages. Regular grammars can describe the forms of tokens, whereas context-free grammars can describe the syntax of whole programming languages, with minor exceptions.

Example: $\langle \text{expr} \rangle \rightarrow \langle \text{var} \rangle + \langle \text{var} \rangle$

$\text{total} = \text{subtotal}_1 + \text{subtotal}_2$

Here $\langle \text{expr} \rangle$ and $\langle \text{var} \rangle$ are non-terminal symbols, while subtotal_1 , subtotal_2 , total and $+$ are the elements used to form the expression.

Backus Naur Form (BNF): Backus Naur form is a formal notation used for specifying the syntax of programming languages. John Backus introduced a formal notation for specifying programming language syntax, which was later modified by Peter Naur for the description of ALGOL 60. The revised method became known as Backus-Naur Form (BNF).

→ BNF became a popular method because it provides a concise and systematic way of describing programming-language syntax. BNF is also called a meta language, because it is a language used to describe another language.

Example: $\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\langle \text{id} \rangle \rightarrow A | B | C$

$\langle \text{expr} \rangle \rightarrow \langle \text{id} \rangle + \langle \text{expr} \rangle /$
 $\langle \text{id} \rangle * \langle \text{expr} \rangle /$
 $(\langle \text{expr} \rangle) / \langle \text{id} \rangle$

for example, the statement $A = B * (A + C)$

$\langle \text{assign} \rangle \Rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$A = \langle \text{expr} \rangle$

$A = \langle \text{id} \rangle * \langle \text{expr} \rangle$

$A = B * (\langle \text{expr} \rangle)$



$A = B * (<id> + <expr>)$

$A = B * (A + <id>)$

$A = B * (A + C)$

• This is called a left most derivation because the leftmost non terminal is replaced first at every step.

• The symbol '|' is used to represent alternatives.

• Non-terminal symbols represent syntactic structures and written inside angular brackets, such as $<expr>$ and $<id>$.

• Terminal symbols are actual symbols or tokens that appear in the program, such as A, B, C, +, * and =.

BNF provides a simple and standard notation for writing those rules.

CFG provides the formal rules for describing the syntax of a programming language.

3 a) Write a short notes on type binding and scoping with example.

Type Binding: Binding is the association between an attribute and an entity, such as associating a variable with its type or value. The time at which a binding takes place is called binding time. Binding and binding times are important concepts in the semantics of programming languages.

Before a variable can be referenced in a program, it must be bound to a data type. The two important aspects of type binding are how type is specified and when the binding takes place.

Types of type binding

1. **Static type binding:** A variable is determined before or during program execution and generally remains fixed.

Example: `var sum = 0;`
`var total = 0.0;`
`var name = "Fred";`

Here, sum is bound to int, total to float, and name to string.

2. **Dynamic type binding:** The type of a variable is determined during run-time and can change during execution.



Example: `list = [10.2, 3.5];`

`list = 47;`

Initially `list` refers to an array of floating-point values. Later it is assigned the integer value 47, so its type can change during execution.

Scoping.

Scope is the range of statements in which a variable is visible. A variable is visible if it can be referenced or assigned in that statement. Scope's rules determine how a particular name is associated with a variable.

Local variable: A variable declared with a program unit/block.

Non local variable: A variable that is visible in a program unit but declared outside it.

Global variable: A special category of non local variable that can be visible to different parts of a program.

Example: `g = 3 #global`
`def sub1():`
`a = 5 #local.`

a) Static Scoping

Static scoping determines the scope of a variable from the textual structure of the program. Therefore, its scope can be determined before program execution.

Example:

```
function big() {  
    var x = 3;  
    function sub2() {  
        var y = x;  
    }  
}
```

b) Global scope

A global variable that is defined after a function can be made visible in the function by declaring it to be external.

Example: `int x = 10; // Global variable.`
`void display() {`
`printf("%d", x);`
`} int main() {`
`display();`
`}`



c) Dynamic scope: The dynamic scoping is based on the calling sequence of subprograms, not on their spatial relationship to each other.

Example: Function big() {
 function sub1() {
 var x = 7;
 } function sub2() {
 var y = x;
 var a = 3;
 }
 var r = 3;
}

3 b) Explain short-circuit evaluation & Guarded commands
 short-circuit evaluation.
 It evaluates an expression is one in which the result is determined without evaluating all of operands and/or operators.

Example:

if ($x \neq 0$ & $y/x > 5$)

if $x \neq 0$ false, $y/x > 5$ is not evaluated.

Guarded commands.

Guarded commands are control structures where each statement has a Boolean condition called a guard.

Syntax

```
if
  condition → statement
[] condition → statement
fi
```

Example: if

$i = 0 \rightarrow \text{sum} := \text{sum} + i$

[] $i > j \rightarrow \text{sum} := \text{sum} + j$

[] $j > i \rightarrow \text{sum} := \text{sum} + k$

fi

if more than one condition is true, one corresponding statement can be selected non-deterministically.

4 a) What is over-loaded operator? Explain with example.



An over loaded operator is an operator that have more than one meaning or operation, depending on the type of operands or the context in which it is used.

→ Operator overloading allows the same operator symbol to perform different operations.

Example

The operator "+" can be used for different purposes

In C++:

```
int a = 10, b = 20;
```

```
count << a + b;
```

Here, + performs arithmetic addition.

For strings:

```
string a = "Hello";
```

```
string b = "World";
```

```
count << a + b;
```

Here, + performs string concatenation.

Thus, the same + operator performs:

- addition for numbers &
- concatenation for strings

4) b) Discuss about control structures, selection statements and iterative statements.

Control structure:

A control structure controls the flow of execution of statements in a program. It provides way to alternative path and repeat statements.

Selection statements.

A selection statement chooses between two or more execution paths.

Types:

- Two way selection: if - else
- Multiple selection: switch



Examples:

Two-way selection

```
if (marks >= 40)
    print("Pass");
```

```
else
    print("Fail");
```

Iterative statements

Iterative statements repeatedly execute a statement or group of statements.

Common types are

- while
- do-while
- for

Example:

```
* for (i=1; i <= 5; i++)
    print(i);
```

the loop executes repeatedly until the specified condition becomes false.

```
* i=1;
```

```
do {
    print(i);
```

```
    i++;
```

```
} while (i <= 5);
```

It will always execute at least once even if the condition is false.

Multiple selection.
switch (choice) {

```
case 1: print("One");
        break;
```

```
case 2: print("Two");
        break;
```

```
}
```



a) Define sub-program. Mention the design issues of sub-program and function.

Subprogram ..

A subprogram is a fundamental building block of a program. It is a collection of statements written to perform a specific task. Subprograms improve readability by emphasizing the logical structure of a program while hiding low-level details. They also support code reuse, saving memory space and coding time.

Design Issues for Subprogram

- (i) If subprogram can be passed as parameters and subprograms can be nested, what is the referencing environment of a passed subprogram?
 - (ii) Can subprograms be overloaded?
 - (iii) Can subprograms be generic?
 - (iv) If nested subprograms are allowed, are closures supported?
 - (v) Are local variables statically or dynamically allocated?
 - (vi) Can subprogram definitions appear inside other subprogram definitions?
 - (vii) What parameter-passing techniques are used?
 - (viii) Are the types of actual parameters checked against the types of formal parameters?
- Design Issues for function.
- (i) Are side effects allowed?
 - (ii) What types of values can be returned?
 - (iii) How many values can be ~~writt~~ returned?



For example, most imperative languages allow functions to return values and may allow side effects through parameters.

b) Explain implementation model and parameter passing. with example.

Parameter passing: Parameter passing methods are the ways in which parameters are transmitted to and/or from called subprograms.

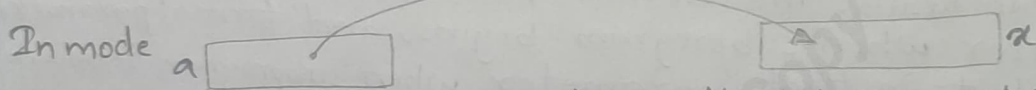
There are two models:

1. Semantic Model of parameter passing.

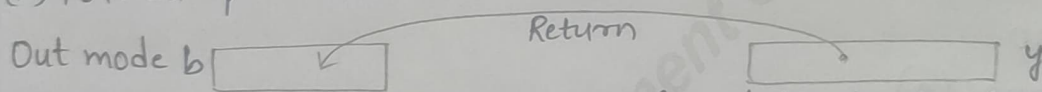
2. Implementation model of parameter passing.

1. Semantic Model of parameter passing.

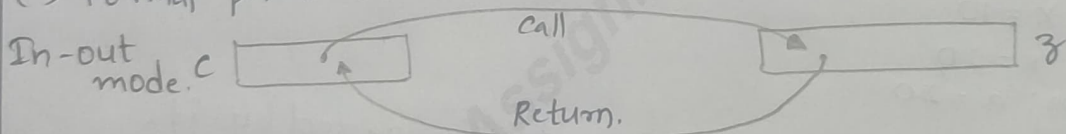
i) Formal parameter can receive data from corresponding actual parameter.



ii) Formal parameter can transmit data to actual parameter.



iii) Formal parameters can receive data & also transmit data



2. Implementation model of parameter passing.

A variety of models have been developed to guide the implementation of three basic parameter transmission modes.

i) Pass-by-Value.

ii) Pass-by-Result.

iii) Pass-by-Value-Result.

iv) Pass-by-Reference.

v) Pass-by-Name.

i) Pass-by-Value: The value of actual parameter is used to initialize the corresponding formal parameter. The formal parameter then acts as local variable in the subprogram.

Ex: void swap(int a, int b) {
 int temp = a;
 a = b;
 b = temp;
}



```
int main() {  
    int c = 10, d = 20;  
    swap(c, d);  
    printf("c = %d, d = %d", c, d);  
    return 0;  
}
```

If `swap(c, d)` is called, the values of `c` and `d` are copied into `a` and `b`. Changes to `a` and `b` do not change `c` and `d`.

2. Pass-by result: No value is transmitted to subprogram initially. The value produced by formal parameter is transmitted back to actual parameter.

```
Ex: void Fixer(out int x, out int y) {  
    x = 17;  
    y = 35;  
} Fixer(out a, out b);
```

After the subprogram finishes: `a = 17, b = 35`

3. Pass-by-Value-Result: The actual parameter is copied to formal parameter when subprogram begins, and formal parameter is copied back to actual parameter when subprogram terminates.

```
Ex: int a = 10  
    Call: change(a)  
    At call: x = 10  
    Inside: x = 20  
    At return: a = 20
```

4. Pass by Reference: The parameter is passed by its address or reference. It is efficient in terms of time and space, but aliases can be created.

```
Ex: void fun(int &first, int &second) {  
    // statements  
} fun(total, total);
```

Here, `first` and `second` refers to the same variable, so they become aliases.

5. Pass by Name: The actual parameters is substituted for corresponding formal parameters. The ALGOL supports pass-by-name.

```
Ex: procedure change(x);  
    begin  
        x := x + 10;  
    end;  
    change(a);
```

Here, `x` represents actual parameter `a` directly. 12